XENTRA DATACLINIC ��� Manual de desarrollo de XENTRA DATACLINIC

Table of Contents

Manual de desarrollo de XENTRA DATACLINIC

Cómo se construyó la aplicación, paso a paso, en qué se basó y por qué.

Versión del software	0.3.0
Fecha del manual	Noviembre de 2026
Repositorio	`github.com/ingjosecastro/xentra-dataclinic`, rama `claude/electronic-health-records-system-1zx3xq`
Público	Desarrolladores, líderes técnicos, auditores de sistemas, interventoría y la gerencia que necesite entender las decisiones

Contenido

1. Propósito y alcance
2. Fundamentos: normas, estándares y buenas prácticas
3. Decisiones de arquitectura y su justificación
4. Preparación del entorno de desarrollo
5. Cronología del desarrollo, fase por fase
6. Diseño de la base de datos
7. Diseño de seguridad
8. Diseño de la inteligencia artificial
9. Interoperabilidad: RIPS, FHIR y RDA
10. Frontend
11. Pruebas y aseguramiento de calidad
12. Migraciones, despliegue e integración continua
13. Cómo extender el sistema
14. Convenciones de código
15. Limitaciones conocidas y trabajo pendiente

1. Propósito y alcance

1.1 Problema que resuelve

Una IPS colombiana necesita en un solo sistema:

1. **Historia clínica electrónica** legalmente válida: integral, secuencial, inalterable y reservada (Resolución 1995 de 1999).
2. **Gestión hospitalaria**: agenda, urgencias, hospitalización, laboratorio, farmacia y facturación.
3. **Reportes obligatorios**: RIPS en JSON como soporte de la factura electrónica (Resolución 2275 de 2023), RDA hacia la IHCE (Resolución 1888 de 2025) y eventos SIVIGILA.
4. **Seguridad verificable** frente a manipulación y ciberataques, que en el sector salud colombiano son frecuentes (ver `docs/10-analisis-mercado.md`).
5. **Autonomía de la IPS**: crear reportes y formularios nuevos sin depender del proveedor.

1.2 Requisitos iniciales del cliente

El cliente pidió: backend en **FastAPI**, frontend en **Vue.js**, base de datos **PostgreSQL** ("postgrest"), seguridad **blockchain**, **RIPS**, **inteligencia artificial para CIE-10**, cumplimiento de la **norma colombiana**, y explicaciones "para principiantes, sin omitir procesos".

1.3 Cómo se interpretaron

Pedido	Interpretación técnica	Por qué
"postgrest"	**PostgreSQL** como base de datos, sin la herramienta PostgREST	PostgREST expone la base como API sin capa intermedia; las reglas clínicas (quién firma, qué es inalterable, auditoría de accesos) quedarían dispersas. Con FastAPI quedan en un solo lugar y probadas
"seguridad blockchain"	**Libro mayor encadenado** (SHA-256 + HMAC) con anclaje externo opcional	Da la propiedad útil de una blockchain (alteración detectable) **sin poner datos de salud en una red pública**, lo que violaría la Ley 1581 de 2012
"IA CIE-10"	Motor local sin internet + Claude opcional, con anonimización y	La IA solo **sugiere**; el médico decide. Nunca se muestran códigos

Pedido	Interpretación técnica	Por qué
	validación contra catálogo	inexistentes
"el mejor del mercado"	Hoja de ruta por fases que cubre los módulos de un HIS completo	Ver sección 5

2. Fundamentos: normas, estándares y buenas prácticas

Cada función del sistema responde a una fuente concreta. Esta es la base de todo el diseño.

2.1 Normatividad colombiana

Norma	Qué exige	Dónde se aplica en el código
Resolución 1995 de 1999 – manejo de la historia clínica	Integralidad, secuencialidad, inalterabilidad, custodia; correcciones con nota fechada	Firma que sella el folio (`services/historia.py`), triggers de PostgreSQL (`core/inmutabilidad.py`), notas aclaratorias inmutables, consecutivo de folio por paciente
Ley 23 de 1981 – ética médica	Reserva de la historia clínica	Control de acceso por roles (`core/deps.py`), registro de cada consulta de la HC en el ledger
Ley 1581 de 2012 y Decreto 1377 de 2013	Datos de salud = sensibles; autorización previa, expresa e informada; derechos del titular	Casilla de autorización con fecha, anonimización antes de usar IA externa (`services/texto.py`), el paciente ve quién consultó su HC (`api/portal.py`), BI solo con datos agregados
Resolución 839 de 2017	Conservación de la HC (mínimo 15 años)	Nada clínico se borra; respaldos diarios con prueba de restauración
Ley 527 de 1999 y Decreto 2364 de 2012	Validez de mensajes de datos y firma electrónica	Firma electrónica = usuario autenticado + hash SHA-256 + bloque en el ledger + QR

Norma	Qué exige	Dónde se aplica en el código
		verificable
Resolución 2275 de 2023	RIPS en JSON como soporte de la factura electrónica; validación en el MUV	`services/rips.py`, `services/facturacion.py`, adaptador MUV en `services/integraciones.py`
Ley 2015 de 2020 y **Resolución 866 de 2021**	Historia clínica electrónica interoperable; elementos de datos	API FHIR R4 (`api/fhir.py`)
Resolución 1888 de 2025	Resumen Digital de Atención (RDA) en FHIR; obligatorio desde el 15/04/2026	`services/rda.py`, cola de envío con reintentos
Resolución 1799 de 2026	RDA de dispensación de medicamentos; plazo hasta el 01/03/2027	`rda_dispensacion()` genera `MedicationDispense` RDA en cada entrega de farmacia
Resolución 5596 de 2015	Triage en 5 categorías; I inmediata, II máximo 30 minutos	`services/hospital.py` (sugerencia y tiempos), cola de urgencias
Resolución 3280 de 2018	Rutas integrales de atención; cursos de vida	Plantillas de control prenatal y crecimiento y desarrollo; dimensión "curso de vida" en el BI
Resolución 256 de 2016	Indicadores de calidad	`api/reportes.py` (oportunidad de citas, triage, estancia, mortalidad)
Resolución 2654 de 2019	Telesalud	Citas de telemedicina con sala de video y consentimiento específico
Resolución 3100 de 2019	Habilitación de servicios	Código de servicio REPS por atención; sedes con código de habilitación
SIVIGILA (INS)	Notificación obligatoria de eventos de interés en salud pública	`services/seguridad_clinica.py` detecta los

Norma	Qué exige	Dónde se aplica en el código
		diagnósticos al firmar

2.2 Estándares técnicos y de terminología

Estándar	Uso	Por qué
HL7 FHIR R4	API de interoperabilidad, RDA	Exigido por la Res. 1888 de 2025 y por la guía de la IHCE; estándar internacional
Guía FHIR RDA v1.0.3 (IHCE, HL7 Colombia)	Perfiles `CompositionAmbulatoryRDA`, `CompositionEmergencyRDA`, `CompositionHospitalizationRDA`, `CompositionPatientStatementRDA`, `MedicationDispenseRDA`; secciones y códigos LOINC	Es la especificación oficial del Ministerio
CIE-10 (formato RIPS de 4 caracteres)	Diagnósticos	Exigido por RIPS; los códigos de 3 caracteres llevan "X" (I10 → I10X)
CUPS, **CUM**, **DIVIPOLA** y tablas de referencia **SISPRO**	Procedimientos, medicamentos, municipios y valores de RIPS	Terminologías oficiales colombianas
LOINC	Signos vitales y secciones del RDA	Exigido por la guía FHIR RDA
UCUM	Unidades de medida en FHIR	Estándar de FHIR
RFC 6238 (TOTP)	Segundo factor de autenticación	Compatible con cualquier app autenticadora
OpenTimestamps	Anclaje del ledger en Bitcoin	Sello de tiempo público y gratuito, sin exponer datos

2.3 Buenas prácticas de seguridad

Referencia	Aplicación
OWASP ASVS / Top 10	Validación estricta de entradas (Pydantic), sin SQL libre, contraseñas

Referencia	Aplicación
	con Argon2id, control de acceso en cada endpoint, cabeceras de seguridad, errores sin detalles técnicos
OWASP Password Storage Cheat Sheet	Argon2id como algoritmo de contraseñas
Principio de mínimo privilegio	La API corre con un usuario de base de datos que no es dueño de las tablas
Defensa en profundidad	API → triggers de BD → ledger con HMAC → anclaje externo
ISO/IEC 27001 y 27799	Referencia para el plan de seguridad y continuidad (no es una certificación del software)

2.4 Análisis del mercado

Se revisaron fuentes públicas sobre los sistemas líderes en Colombia (Dinámica Gerencial, Zeus Salud, Panacea, SISMA, HeOn) y sobre incidentes del sector. No se encontraron quejas verificables de esos productos; sí se documentaron problemas del sector: ransomware en hospitales, falta de interoperabilidad, hallazgos de auditoría en historias clínicas y nuevas obligaciones (RDA). Esas brechas guiaron los módulos de continuidad, soporte, BI y RDA. Detalle y fuentes: `docs/10-analisis-mercado.md`.

---

3. Decisiones de arquitectura y su justificación

3.1 Vista general

```
Navegador ──HTTPS──► Caddy (HTTPS automático, solo producción)
                          │
                          ▼
                    Nginx (contenedor "frontend"): sirve Vue y reenvía
/api y /fhir
                          │
                          ▼
                    FastAPI (contenedor "backend") ──► servicios externos
(simulados por defecto):
                      - autenticación y roles          Claude (IA), DIAN
(proveedor),
                      - reglas clínicas                MUV (FEV-RIPS),
IHCE (RDA),
                      - ledger, RIPS, RDA, BI          OpenTimestamps,
Jitsi
                          │
                          ▼
                    PostgreSQL 16 ("db"): triggers de inmutabilidad,
```

```
usuario de aplicación sin privilegios
                    ▲
                  "respaldo": pg_dump diario + prueba de restauración
semanal
```

3.2 Tecnologías elegidas

Componente	Elección	Alternativas consideradas	Razón de la elección
Lenguaje backend	Python 3.13	Java (Spring), C# (.NET), Node (NestJS)	Pedido del cliente; ecosistema de IA y datos más fuerte; curva de aprendizaje baja
Framework API	FastAPI	Django REST, Flask	Validación automática con tipos, documentación Swagger automática, alto rendimiento
ORM	SQLAlchemy 2	Django ORM, SQL a mano	Estándar en Python; soporta PostgreSQL y SQLite (pruebas rápidas)
Migraciones	Alembic	create_all	Cambios de esquema versionados y verificables en CI
Validación	Pydantic v2	Marshmallow	Integrado con FastAPI
Base de datos	PostgreSQL 16	MySQL, MongoDB	Transacciones, integridad referencial, triggers y JSONB; NoSQL no da garantías de integridad para la HC
Frontend	Vue 3 + TypeScript + Vite + Pinia	React, Angular	Pedido del cliente; TypeScript detecta errores antes de

Componente	Elección	Alternativas consideradas	Razón de la elección
			producción
Gráficos	SVG propio	Chart.js, ECharts	Sin dependencias externas y sin enviar datos a terceros
PDF	fpdf2 + segno (QR)	ReportLab, WeasyPrint	Livianas, Python puro
Excel	openpyxl	—	Estándar para .xlsx
Contraseñas	Argon2id (argon2-cffi)	bcrypt	Recomendado por OWASP; passlib está sin mantenimiento
Tokens	JWT (PyJWT), 30 min	Sesiones en servidor	API sin estado; sesión deslizante con renovación
Contenedores	Docker Compose	Kubernetes	Simple para una IPS; un solo comando
HTTPS	Caddy	Nginx + certbot	Certificados automáticos sin configuración
IA	Motor local + SDK oficial de Anthropic (Claude)	Solo nube	Funciona sin internet y sin enviar datos; Claude es opcional

3.3 Organización del código

Regla de oro: las rutas HTTP (`app/api/`) solo reciben la petición, verifican permisos y llaman a la lógica de negocio (`app/services/`), que no depende de HTTP y por eso se puede probar sola.

```
backend/app/
├── main.py              Arranque, middlewares (seguridad, código de
error), registro de rutas
├── models.py            Tablas núcleo: usuarios, pacientes,
atenciones, diagnósticos, ledger, CIE-10
├── models_ext.py        Tablas hospitalarias, facturación, RDA, BI,
módulos, soporte
├── schemas.py           Validación de entrada/salida del núcleo
├── schemas_ext.py       Validación de los módulos hospitalarios
```

```
├── seed.py              Administrador, catálogos de muestra, datos
demo
├── core/                config, database, security, deps (roles),
inmutabilidad (triggers)
├── api/                 21 archivos de rutas, uno por módulo
├── services/            20 archivos de lógica: ledger, rips, rda,
fhir, ia, bi, pdf, mfa…
├── cargar_cie10.py, cargar_catalogo.py, anclar.py, procesar_rda.py
(scripts de consola)
backend/alembic/         Migraciones 0001 (esquema inicial) y 0002
(RDA, BI, módulos, soporte)
backend/tests/           51 pruebas automáticas
frontend/src/            views (23 pantallas), components, stores
(sesión), router, api (cliente HTTP)
db/                      Usuario de BD sin privilegios y script de
respaldos
```

---

4. Preparación del entorno de desarrollo

4.1 Programas necesarios

Programa	Versión usada	Para qué
Git	2.x	Control de versiones
Python	3.13	Backend
Node.js	22 LTS	Frontend
PostgreSQL	16	Base de datos (o Docker)
Docker Desktop	Actual	Ejecutar todo con un comando
Visual Studio Code	Actual	Editor, con extensiones *Python* y *Vue - Official*

4.2 Clonar y preparar el backend

```
git clone https://github.com/ingjosecastro/xentra-dataclinic.git
cd xentra-dataclinic
git checkout claude/electronic-health-records-system-1zx3xq
cd backend
python -m venv .venv
# Windows: .venv\Scripts\activate      Mac/Linux: source
.venv/bin/activate
pip install -r requirements-dev.txt
```

Cree `backend/.env` con al menos:

```
DATABASE_URL=postgresql+psycopg://xentra:clave@localhost:5432/
xentra_hce
SECRET_KEY=texto_aleatorio_de_mas_de_32_caracteres
```

```
LEDGER_HMAC_KEY=otro_texto_aleatorio_de_mas_de_32_caracteres
ADMIN_PASSWORD=Admin12345*
```

Arranque: `uvicorn app.main:app --reload` → documentación de la API en `http://localhost:8000/docs`.

En desarrollo `AUTO_CREAR_TABLAS=true` (valor por defecto): las tablas y los triggers se crean al arrancar.

4.3 Preparar el frontend

```
cd frontend
npm install
npm run dev        # http://localhost:5173 (reenvía /api al puerto 8000)
npm run build      # verifica los tipos (vue-tsc) y compila
```

4.4 Ejecutar las pruebas

```
cd backend
pytest                                                    # SQLite temporal (segundos)
TEST_DATABASE_URL=postgresql+psycopg://usuario:clave@localhost:5432/xentra_test pytest   # PostgreSQL + triggers
```

Nunca apunte las pruebas a la base de producción: las pruebas borran y recrean las tablas.

---

5. Cronología del desarrollo, fase por fase

El sistema se construyó en incrementos, cada uno **probado y con su propio commit**. Así cualquier cambio se puede rastrear y revertir.

Fase 0 – MVP (commit 19a3e64)

Objetivo: demostrar el núcleo legal de la historia clínica.

Pasos realizados:

1. **Estructura del repositorio** (`backend/`, `frontend/`, `docs/`), `.gitignore` que excluye `.env`, y `.env.example` documentado.
2. **Configuración central** (`core/config.py`): todo se lee de variables de entorno, nunca hay claves en el código.
3. **Conexión a la BD** (`core/database.py`) con SQLAlchemy 2.
4. **Seguridad básica** (`core/security.py`): Argon2id, política de contraseñas y JWT.
5. **Modelos** (`models.py`): usuarios, pacientes con los campos de RIPS, atenciones en formato SOAP, diagnósticos, procedimientos, prescripciones, notas aclaratorias, ledger y CIE-10.
6. **Ledger** (`services/ledger.py`): cadena SHA-256 + firma HMAC + verificación completa; bloqueo consultivo de PostgreSQL para que dos peticiones simultáneas no rompan la cadena.

7. **Firma del folio** (`services/historia.py`): hash del contenido clínico sellado en el ledger.
8. **Triggers de inmutabilidad** (`core/inmutabilidad.py`): PostgreSQL rechaza modificar folios firmados y el ledger.
   - *Lección aprendida:* el primer intento fallaba en PostgreSQL porque la firma hacía dos `UPDATE` y el segundo ya chocaba con el trigger. Se reordenó: primero se sella en el ledger y luego se marca firmado en un único `UPDATE`.
9. **Catálogo CIE-10** de muestra (~220 códigos frecuentes) y motor local de sugerencias (sinónimos coloquiales colombianos + TF-IDF + filtros de sexo y edad).
10. **IA con Claude** opcional, con anonimización y validación contra el catálogo.
11. **RIPS** de consultas y procedimientos según la Res. 2275 de 2023.
12. **API** (autenticación, pacientes, atenciones, CIE-10, RIPS, auditoría) con control de roles.
13. **Frontend Vue**: login, pacientes, atención SOAP con asistente IA, folio firmado con verificación, RIPS y auditoría.
14. **Docker Compose** con Nginx.
15. **20 pruebas** en SQLite y PostgreSQL, y prueba completa en navegador con Playwright.
16. **Documentación para principiantes** (8 documentos).

Fase 1 – Base ampliada (commit `980a1a0`)

Paso	Qué se hizo	Por qué
1	Modelos extendidos (`models_ext.py`) para todos los módulos hospitalarios	Separar el núcleo de la gestión
2	Catálogos de referencia (CUPS, DIVIPOLA, EAPB, tablas RIPS) con cargador	Los códigos RIPS deben salir de tablas oficiales, no escribirse a mano
3	Sedes	Multi-sede con código de habilitación propio
4	MFA TOTP implementado según RFC 6238, sin dependencias	Proteger cuentas aunque se filtre la contraseña
5	Sesión deslizante (renovación) y token intermedio de MFA que no sirve para usar la API	Seguridad sin incomodar al usuario
6	PDF con QR y verificación pública sin datos clínicos	Una farmacia o EPS puede verificar un documento sin ver información sensible
7	Alergias con reactividad	Seguridad del paciente y

Paso	Qué se hizo	Por qué
	cruzada, interacciones y detección SIVIGILA	salud pública
8	Plantillas de historia clínica	Registros completos y homogéneos

Fase 2 – Agenda y facturación (commit `5341a93`)

1. **Agenda**: cruces de horario por profesional y paciente, disponibilidad, transiciones de estado controladas, reprogramación y recordatorios.
   - *Decisión:* todas las fechas se guardan en **UTC** y se muestran en hora de Colombia, para evitar errores entre SQLite y PostgreSQL y con el cambio de zona.
2. **Telemedicina**: sala Jitsi con nombre aleatorio por cita.
3. **Facturación**: armado de la cuenta desde atenciones firmadas y hospitalizaciones egresadas; no permite facturar dos veces un servicio.
4. **Pre-auditoría**: reglas contra las glosas más frecuentes (ítems sin valor, EPS distinta, falta de autorización, síntoma marcado como confirmado, duplicidades).
5. **RIPS completo**: urgencias, hospitalización y medicamentos dispensados.
6. **Adaptadores DIAN y MUV** con modo simulado y modo real, porque cada proveedor DIAN tiene su propia API y la IPS necesita probar sin enviar nada.

Fase 3 – Hospital (commit `545bc4e`)

1. **Triage** con sugerencia por signos vitales y palabras clave, y tiempos de la Res. 5596 de 2015.
2. **Hospitalización**: camas, ingreso, evoluciones (solo inserción), órdenes con alertas que exigen confirmación, kárdex, balance de líquidos, traslados y egreso con epicrisis sellada.
3. **Farmacia**: lotes, vencimientos y dispensación **FEFO** (primero lo que vence antes; nunca lotes vencidos).
4. **Laboratorio**: órdenes, resultados con marca automática de valores anormales y validación que sella.
5. **Consentimientos** con firma manuscrita en pantalla.
6. Nuevos triggers de "solo inserción" y de registros sellados.

Fase 4 – Interoperabilidad y paciente (commit `75b90cf`)

1. **FHIR R4**: Patient, Practitioner, Encounter, Condition, MedicationRequest, Procedure, Observation, AllergyIntolerance y `$everything`.
2. **Portal del paciente**: historia, citas (con teleconsulta), resultados, PDF, **quién consultó su historia** y descarga completa en FHIR (portabilidad).
3. **Resumen clínico** estructurado (local) y narrativo (Claude, opcional).
4. **Indicadores** y reporte SIVIGILA en CSV.
5. **Anclaje del ledger** en OpenTimestamps.

Fase 5 – Frontend de todos los módulos (commit `5ef23d5`)

Pantallas de agenda, urgencias, hospitalización, laboratorio, farmacia, facturación, indicadores, portal, cuenta (MFA) y verificación pública. Se agregaron componentes de dictado por voz, firma manuscrita y listas desde catálogos, además del cierre por inactividad (15 minutos).

Fase 6 – Infraestructura de producción (commit `0bee541`)

1. **Alembic**: migración inicial generada contra PostgreSQL real, con triggers y permisos incluidos.
2. **Usuario de BD sin privilegios** (`db/init/01-usuario-aplicacion.sh`). Se comprobó que no puede alterar el ledger ni desactivar triggers.
3. **Servicio `migrate`**: aplica las migraciones con el usuario dueño y termina.
4. **HTTPS** con Caddy (`docker-compose.prod.yml`).
5. **Respaldos** diarios con rotación.
6. **CI en GitHub Actions**: pruebas en SQLite y PostgreSQL, `alembic check` y compilación del frontend.

Fase 7 – Compatibilidad con Windows (commit `b1f308e`)

`.gitattributes` fuerza finales de línea LF en scripts `.sh`. Sin esto, al clonar en Windows los scripts del contenedor fallan.

Fase 8 – Brechas del mercado (commits `c214be2` y `341b0ba`)

Resultado del análisis de mercado (sección 2.4):

1. **RDA** (Res. 1888 de 2025) con cola de envío y reintentos.
2. **Constructor de reportes BI** sin SQL libre, con exportación a Excel.
3. **Módulos dinámicos** creados por la IPS sin programar.
4. **Soporte**: código de error trazable, mesa de ayuda con ANS y base de conocimiento.
5. **Continuidad**: estado del sistema, avisos, PDF de contingencia, Nginx que sigue sirviendo la aplicación si la API cae y prueba semanal de restauración.
    - *Lección aprendida:* la migración 0001 importaba el SQL de triggers "vivo"; al agregar una tabla nueva, la 0001 dejó de funcionar. Se separó el SQL por versión (`SQL_V1`, `SQL_V2`) para que cada migración quede congelada.

Fase 9 – Alineación con la guía oficial del RDA (commits `cdfb3a7` y `2a044f5`)

Con acceso a la guía oficial (vulcano.ihcecol.gov.co) y al texto de la resolución se verificaron:

- URL canónica de los perfiles: `https://fhir.minsalud.gov.co/rda/StructureDefinition/`.
- Secciones y códigos LOINC exactos por perfil.
- Que la dispensación se envía como `MedicationDispense` y no como documento.
- La fecha de obligatoriedad: 15/04/2026 (artículo 5).

La validación estructural detectó dos referencias rotas antes de que se enviaran, y se corrigieron.

Fase 10 – Visibilidad (commit `5caf5ae`)

El indicador de seguridad blockchain se muestra a todo el personal en el inicio; el menú pasó a llamarse "Seguridad blockchain" y se creó un validador de RIPS independiente.

---

6. Diseño de la base de datos

6.1 Principios

1. **UUID** como identificador de registros clínicos: no revela cuántos pacientes hay y permite integrar sistemas.
2. **UTC** en todas las fechas.
3. **JSONB** solo para datos semiestructurados (signos vitales, antecedentes, resultados, campos de módulos dinámicos).
4. **Nada clínico se borra**. Los borradores sí, porque no tienen valor legal.
5. **Restricciones en la propia BD** (únicos, llaves foráneas, triggers), no solo en la aplicación.

6.2 Tablas por área

Área	Tablas
Identidad	`usuarios`, `sedes`
Pacientes e HC	`pacientes`, `atenciones`, `diagnosticos`, `procedimientos`, `prescripciones`, `notas_aclaratorias`, `consentimientos`
Catálogos	`cie10`, `catalogos`
Agenda / urgencias	`citas`, `triages`
Hospitalización	`camas`, `hospitalizaciones`, `evoluciones`, `ordenes_medicas`, `administraciones_medicamento`, `balance_liquidos`
Farmacia	`productos`, `lotes`, `movimientos_inventario`
Laboratorio	`ordenes_servicio`
Facturación	`facturas`, `factura_items`
Interoperabilidad	`rda_envios`
Seguridad	`ledger_bloques`, `anclajes_ledger`
BI y módulos	`reportes_guardados`, `modulos_dinamicos`, `registros_modulo`
Soporte	`tickets`, `avisos_sistema`

6.3 Triggers de inmutabilidad (15)

Tipo	Tablas	Regla
Inmutable siempre	`ledger_bloques`, `notas_aclaratorias`	Ni `UPDATE` ni `DELETE`
Sellado al firmar	`atenciones` y sus hijos (`diagnosticos`, `procedimientos`, `prescripciones`)	Bloqueados cuando la atención está firmada
Solo inserción	`evoluciones`, `administraciones_medicamento`, `consentimientos`, `balance_liquidos`, `movimientos_inventario`, `anclajes_ledger`	Se agregan, nunca se cambian
Sellado por estado	`hospitalizaciones` (egresada), `ordenes_servicio` (validada), `registros_modulo` (sellado)	Bloqueados tras el cierre

7. Diseño de seguridad

7.1 Capas

1. **Red**: solo se publica el frontend; la BD queda en la red interna de Docker.
2. **Transporte**: HTTPS con HSTS (Caddy).
3. **Autenticación**: Argon2id, política de complejidad, bloqueo a los 5 intentos durante 15 minutos, MFA TOTP, JWT de 30 minutos con renovación y cierre a los 15 minutos de inactividad.
4. **Autorización**: cada endpoint declara sus roles (`requiere_rol`). El paciente solo accede al portal.
5. **Validación**: Pydantic en todas las entradas. El BI no admite SQL libre (lista blanca de dimensiones y medidas).
6. **Base de datos**: triggers y usuario sin privilegios.
7. **Auditoría**: ledger con HMAC (llave fuera de la BD) y anclaje externo.
8. **Navegador**: cabeceras de seguridad, `Cache-Control: no-store`, sesión en `sessionStorage`.
9. **Errores**: nunca se muestran detalles técnicos; se entrega un código para soporte.

7.2 El ledger en detalle

Cada bloque guarda: id, fecha, usuario, IP, acción, entidad, `hash_datos` (huella de los datos, nunca los datos), `hash_anterior`, `hash` y `firma`. La firma es `HMAC(LEDGER_HMAC_KEY, hash)`.

- Alterar un bloque rompe todos los siguientes.
- Recalcular la cadena exige la llave, que no está en la BD.
- El anclaje externo prueba que ni el dueño del servidor reescribió la historia.

---

8. Diseño de la inteligencia artificial

Función	Técnica	Salvaguardas
Sugerencia CIE-10 local	Diccionario de sinónimos coloquiales + TF-IDF sobre las descripciones oficiales + filtro de sexo y edad	Solo códigos del catálogo
Sugerencia CIE-10 con Claude	Respuesta en JSON estructurado; códigos candidatos del motor local como contexto	Texto anonimizado; se descartan códigos inexistentes o no pertinentes; si falla, se usa el motor local
Resumen clínico	Extracción determinista; narrativa opcional con Claude	Datos anonimizados; aviso de verificación
Triage	Reglas sobre signos vitales y palabras clave	El nivel final lo decide el profesional
Alertas	Tablas de alergias cruzadas e interacciones	Tablas de apoyo, no exhaustivas
Pre-auditoría	Reglas de glosa	Informa; solo bloquea errores evidentes

Principio rector: la IA apoya y el profesional decide. Cada diagnóstico guarda si fue sugerido por IA, para poder medir la calidad de las sugerencias.

---

9. Interoperabilidad: RIPS, FHIR y RDA

9.1 RIPS (Res. 2275 de 2023)

- Se generan desde las facturas (o desde el validador independiente).
- Incluyen consultas, procedimientos, urgencias, hospitalización y medicamentos dispensados.

- La validación previa revisa diagnóstico principal, formato CIE-10, sexo, DIVIPOLA, egresos y causa de muerte.
- Pendientes: `recienNacidos` y `otrosServicios`.

9.2 FHIR R4

API de lectura en `/fhir` con identificadores de documento colombianos, CIE-10 en formato internacional (I10X → I10) y en formato RIPS, y LOINC para signos vitales.

9.3 RDA (Res. 1888 de 2025 y 1799 de 2026)

Tipo	Perfil	Cuándo se genera
Consulta externa	`CompositionAmbulatoryRDA`	Al firmar una atención de consulta
Urgencias	`CompositionEmergencyRDA`	Al firmar una atención de urgencias (incluye triage)
Hospitalización	`CompositionHospitalizationRDA`	Al dar egreso
Paciente	`CompositionPatientStatementRDA`	Bajo demanda
Dispensación	`MedicationDispenseRDA`	En cada entrega de farmacia (uno por lote)

Flujo: generar → validar estructura → encolar → enviar (simulado o real) → reintentar con espera exponencial (2^n minutos, hasta 8 intentos).

La atención clínica **nunca se bloquea** por una falla de la IHCE.

---

10. Frontend

- **Enrutador con roles**: cada pantalla declara quién puede verla. El paciente solo ve el portal.
- **Estado de sesión** (Pinia): token, usuario, MFA, inactividad y renovación.
- **Cliente HTTP** (axios): agrega el token, cierra la sesión ante un 401 y traduce los errores a mensajes claros con el código de soporte.
- **Componentes**: selector CIE-10, listas desde catálogos, dictado por voz (Web Speech API), firma manuscrita (canvas) y gráfico SVG.
- **Accesibilidad básica**: etiquetas en los formularios, contraste y diseño adaptable a celular.

---

11. Pruebas y aseguramiento de calidad

11.1 Pruebas automáticas (51)

Archivo	Qué cubre
`test_unitarias.py`	Formato CIE-10, normalización, anonimización, alertas de signos vitales
`test_flujo.py`	Flujo completo de HC, RBAC, bloqueo por intentos, contraseñas débiles, manipulación directa de la BD
`test_ia_claude.py`	Claude simulado: descarta códigos inventados; ante una falla usa el motor local
`test_bloque1.py`	MFA (vector RFC 6238), sesión, catálogos, sedes, alertas, PDF, verificación pública
`test_bloque2.py`	Agenda, telemedicina, facturación, pre-auditoría, DIAN/MUV simulados, RIPS de urgencias
`test_bloque3.py`	Triage, hospitalización completa, farmacia FEFO, laboratorio, consentimientos, triggers
`test_bloque4.py`	FHIR, portal (incluye que no vea datos ajenos), resumen IA, indicadores, anclaje
`test_bloque5.py`	RDA (perfiles, secciones, dispensación), BI, módulos dinámicos, soporte, contingencia, blockchain visible

11.2 Pruebas de punta a punta

Con Playwright se recorrieron en un navegador real los flujos de cada rol: agenda → atención → urgencias → hospitalización → facturación → indicadores → módulos → soporte.

11.3 Política

Ningún bloque se dio por terminado sin las pruebas en verde **en SQLite y en PostgreSQL**, con el frontend compilando sin errores de tipos.

---

12. Migraciones, despliegue e integración continua

- **Migraciones**: `alembic revision --autogenerate -m "..."`, luego revisar el archivo, `alembic upgrade head` y `alembic check`. Si se agrega un trigger, se crea

un nuevo bloque `SQL_Vn` en `core/inmutabilidad.py`; nunca se modifica uno anterior.

- **Despliegue**: `docker compose up -d --build` en desarrollo, y `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` en producción.
- **CI** (`.github/workflows/ci.yml`): pruebas en SQLite y PostgreSQL, migraciones (aplicar, `check` y revertir) y compilación del frontend.

---

13. Cómo extender el sistema

13.1 Sin programar

- **Formularios nuevos**: menú *Módulos adicionales*.
- **Reportes nuevos**: menú *Inteligencia de negocio*.
- **Catálogos oficiales**: `python -m app.cargar_catalogo`.

13.2 Programando un módulo nuevo

1. Modelo en `models_ext.py` y esquemas en `schemas_ext.py`.
2. Lógica en `services/mi_modulo.py`.
3. Rutas en `api/mi_modulo.py`, con `Depends(requiere_rol(...))` y `registrar_evento(...)` en cada acción relevante.
4. Registrar el router en `main.py`.
5. Si debe ser inmutable, agregar el trigger en un nuevo `SQL_Vn`.
6. Migración Alembic.
7. Pruebas en `tests/`.
8. Pantalla en `frontend/src/views/`, ruta con `meta.roles` y enlace en el menú de `App.vue`.
9. Si sus datos deben analizarse, agregar un `Dataset` en `services/bi.py`.

---

14. Convenciones de código

- Código, mensajes y documentación en **español**; identificadores sin tildes.
- Fechas en UTC.
- Nunca registrar datos clínicos en los logs.
- Nunca subir `.env`, respaldos ni datos reales.
- Commits pequeños con mensaje descriptivo; rama → pull request → CI en verde → revisión → merge.

---

15. Limitaciones conocidas y trabajo pendiente

Tema	Estado
Catálogos oficiales completos (CIE-10, CUPS, CUM, tablas SISPRO)	Se cargan con los scripts; el sistema trae muestras

Tema	Estado
Integración real con DIAN	Adaptador listo; ajustar a la API del proveedor contratado
MUV e IHCE en modo real	Requieren las credenciales de la IPS; ruta de "Enviar Dispensación" por confirmar
RIPS `recienNacidos` y `otrosServicios`	Pendientes
Firma digital con certificado ONAC	Pendiente de adquirir el certificado
Docker Compose en el equipo final	Validado en sintaxis; cada componente probado por separado
Pentest, ISO 27001, políticas Ley 1581	Actividades de la institución

16. Referencias

- Ministerio de Salud y Protección Social – normatividad citada en la sección 2.1.
- Guía FHIR RDA v1.0.3 – https://vulcano.ihcecol.gov.co/
- Manual de Operaciones de Interoperabilidad IHCE v1.3 – https://www.minsalud.gov.co/ihce/Manuales/Manual_de_operaciones_interoperabilidad_IHCE_V1_3.pdf
- Resolución 1888 de 2025 – https://www.minsalud.gov.co/Normatividad_Nuevo/Resolucion%20No%201888%20de%202025.pdf
- HL7 FHIR R4 – https://hl7.org/fhir/R4/
- OWASP ASVS – https://owasp.org/www-project-application-security-verification-standard/
- RFC 6238 (TOTP) – https://www.rfc-editor.org/rfc/rfc6238
- OpenTimestamps – https://opentimestamps.org/
- Documentación interna: `docs/01` a `docs/11`.